READ FIRST

From loose parts to a working lab demo

One guide for the exact LPC1768 trainer, three-resistor echo divider, LCD, and buzzer in your supplied references.

You do **not** paste code into the board. You save **main.c** in a Keil project, build a **.hex** file, send that file to the board with **Flash Magic**, then put the board in **RUN** mode.

The four stages you will complete



Two checks need the instructor
Have lab staff identify the physical CN6 pin numbers and confirm the supply / divider pulse voltage. The supplied documents do not establish the physical pin-1 location on your particular board.

Pages	What you do
2-3	Learn the parts and locate the labelled board connectors.
4-6	Build the divider and connect every sensor, LCD, and buzzer wire.
7-9	Set jumpers, connect the PC, and build the Keil project.
10-12	Check supplies, program the board, and verify ECHO before connecting P2.1.
13-16	Get output, test states, solve common problems, and use the final checklist.
17-21	Complete main.c listing; an exact main.c copy is also embedded in the PDF.

Use this only as a bench prototype. Neither this firmware nor these exact connections have been tested on your physical kit. Do not use this project for real vehicle collision protection.

STEP 01 / PREPARE

Put these items on the bench

Do not turn on the kit yet. You will work with power disconnected through the wiring steps.

Item	What to look for / quantity
Trainer board	ALS-SDA-ARMCTXM3-01, LPC1768 piggyback already installed, 12 MHz crystal.
Sensor	One HC-SR04 with VCC, TRIG, ECHO, GND printed beside its pins.
Breadboard	One conventional board with numbered rows and side power rails.
Resistors	Two 1 kΩ (R1/R2) and one 2.2 kΩ (R3). Confirm values with an unpowered resistance test.
Jumper wires	At least eight jumpers plus spares: seven mandatory leads on page 6, with extras for rail bridging if needed.
Ribbon cables	Two straight-through 10-pin FRC cables, usually supplied with the kit.
PC and software	Lab Windows PC with Keil µVision, LPC1768 device support, Flash Magic, and USB-serial driver.
Serial and power	Lab USB-to-serial adapter, grey RS232 cable, and separate lab 5 V power cable.
Test equipment	Multimeter; instructor-operated oscilloscope or suitable analyzer for the ECHO high pulse.
If CN6 is recessed	Compatible 26-pin IDC breakout. Do not wedge loose wires into contacts that do not fit.

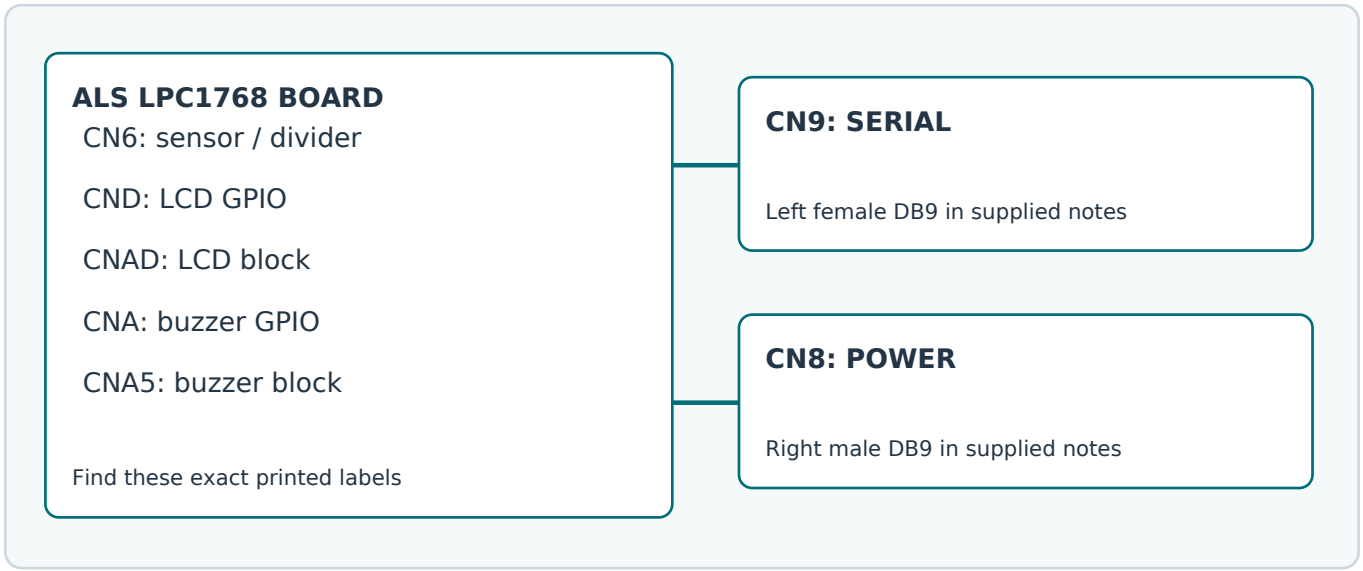
Words you will see

Word	Plain meaning
GPIO / P2.0	A microcontroller signal. P2.0 means port 2, bit 0; it is not a connector number.
CN6 pin 1	A physical contact on the connector labelled CN6 on the board.
GND / VCC	Common electrical reference / sensor positive supply.
Jumper short / open	Plastic cap joins the specified pins / the pins are not joined.
FRC ribbon / HEX	Flat cable with multi-pin ends / compiled program file that Flash Magic sends to the board.

STEP 02 / LOCATE

Find the labels before the pins

This diagram is a connection map. It is not a photo, a board placement drawing, or a physical pin-orientation guide.



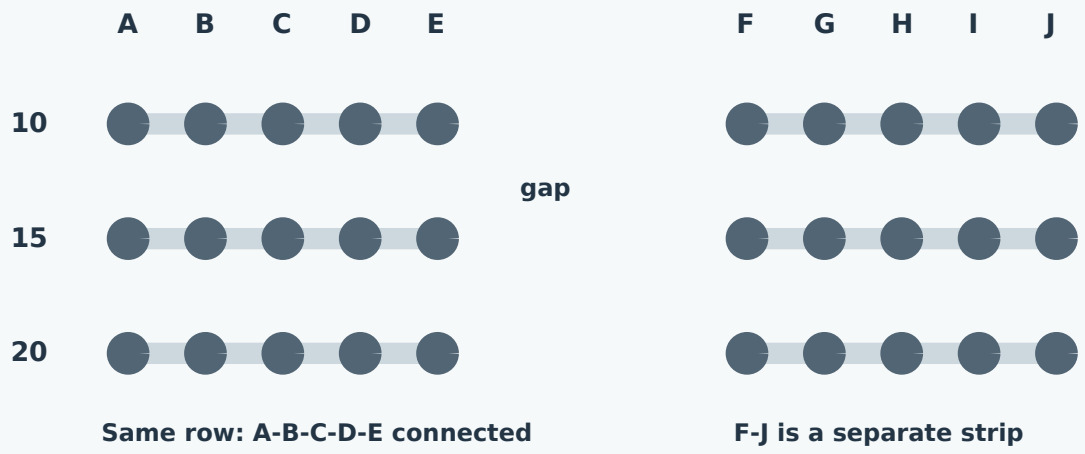
Locate	How you identify it
CN6	26-pin NIFC boxed header. Do not confuse with CND (10-pin), CN5 (JTAG), or CN7 (trace).
CN6 pins 1 / 2 / 25 / 26	Ask staff to confirm physical numbering from the kit drawing / pin-1 marker. Mark nearby tape.
CND + CNAD	Two separate 10-pin connectors; the LCD ribbon will join them.
CNA + CNA5	Two separate 10-pin connectors; the buzzer ribbon will join them. CNA1 is the LED block.
JP1, JP2, JP3, JP5, JP6, JP16	Find the small jumper caps by their printed labels. Do not move unlabelled caps.
SW21 / SW1 / POT3	Two-position programming DIP switch / reset button / LCD contrast adjustment.

Do not guess pin orientation
A generic 2 x 13 drawing cannot tell you which end is pin 1 on this board. If you cannot securely identify and reach pins 1, 2, 25, 26, stop and ask the instructor. Use connector labels, not left/right alone.

STEP 03 / BREADBOARD

Understand the connected holes

All component placement below uses the A-E half of the breadboard. Rows 10, 15, and 20 are examples you can label with tape.



1. Find the row numbers and letters on your breadboard. Check continuity with the kit unpowered if the layout differs.

2. Use rows **10, 15, and 20 on A-E**. Different numbered rows must be electrically separate.

3. Find the side rails. Mark the intended rail **+5 V** and the other **GND**. Check for breaks halfway along a rail. Use the same continuous section for this circuit.

4. Confirm two resistors measure 1 kΩ and one measures 2.2 kΩ. Resistors have no positive/negative orientation.

5. If the sensor is plugged directly into holes, place each of its four pins in a different numbered row. For this guide, connecting its labelled pins with wires is easier.

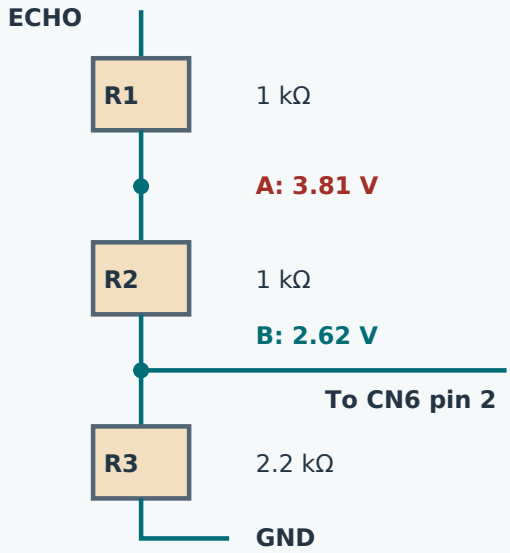
Common beginner mistake
A resistor between A10 and E10 has both ends on the same electrical strip, so it is bypassed. R1 must go between rows 10 and 15, not two holes in row 10.

STEP 04 / DIVIDER

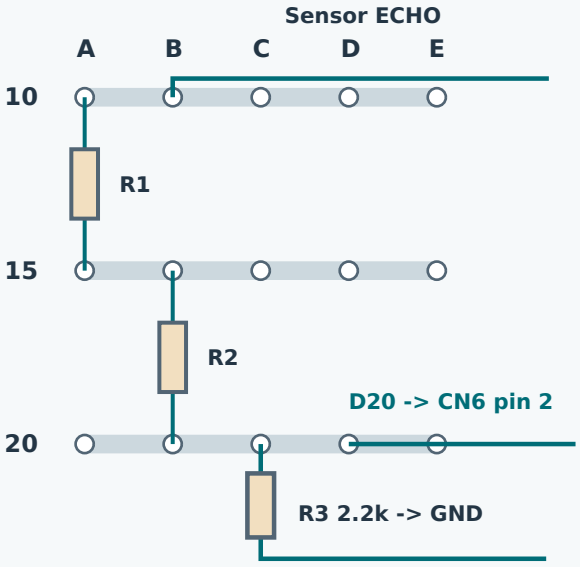
Install all three resistors

Board OFF and power cable unplugged. This circuit reduces the sensor ECHO pulse before it reaches the microcontroller.

Circuit (logical)



Exact hole placement: A-E



Place	Exact two endpoints	What it does
R1 = 1 kΩ	A10 to A15	ECHO input to Node A
R2 = 1 kΩ	B15 to B20	Node A to Node B
R3 = 2.2 kΩ	C20 to the GND rail	Node B to ground

The A/B/C letters simply select different holes on the same strip. Row 15 joins A15 to B15. Row 20 joins B20, C20, and D20.

The only ECHO output for the MCU is Node B
The future P2.1 lead must come from **D20 / row 20**. Do not use row 15 (Node A) or the sensor ECHO pin directly. At 5 V input, Node B = 5 x 2.2 / 4.2 = about 2.62 V.

STEP 05 / ALL SENSOR WIRES

Connect the sensor and rails

Keep power disconnected. Prepare W1-W7 below; W8 is an optional extension. Two ends deliberately remain disconnected until the electrical checks.

Wire	From	To	Initial state
W1	CN6 pin 26	Breadboard GND rail	Connect
W2	HC-SR04 GND	Same GND rail	Connect
W3	CN6 pin 25	Breadboard +5 V rail	Connect; JP1 short later
W4	HC-SR04 VCC	Same +5 V rail	Leave sensor VCC end loose
W5	HC-SR04 TRIG	CN6 pin 1 / P2.0	Connect directly
W6	HC-SR04 ECHO	B10 / row 10	Connect to R1 input
W7	D20 / Node B	CN6 pin 2 / P2.1	Leave CN6 pin 2 end loose
W8	R3 ground end, if not already in rail	GND rail	Only if needed; direct resistor-to-rail placement needs no extra wire

There are **seven mandatory jumper wires (W1-W7)** in this arrangement. W8 is optional if R3 reaches the ground rail directly; take at least eight jumpers plus spares. Keep loose ends isolated so they cannot touch other contacts.



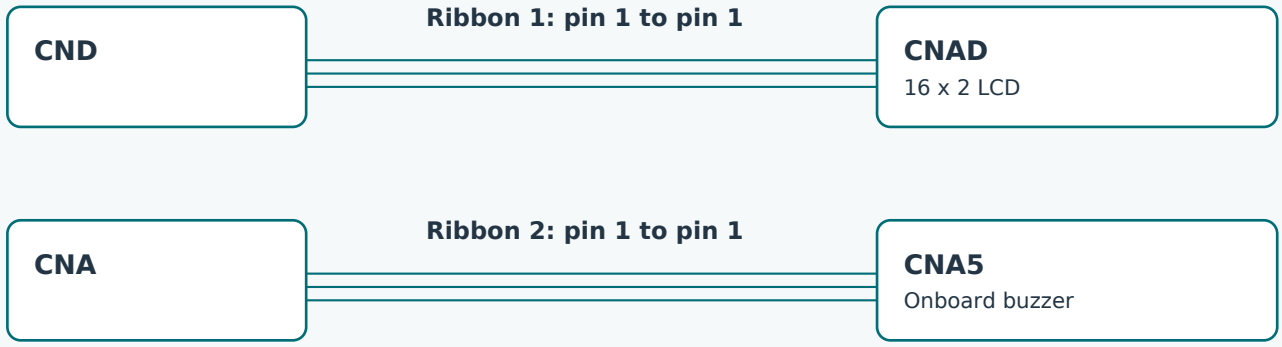
Use the sensor PCB labels; this is not a physical pin-order picture.

Check sensor labels, not a memorized view
The reference photos show VCC/TRIG/ECHO/GND from top to bottom with front transducers facing you and pins on the left; the rear view reverses that order. A rotated module changes the view. The printed labels win.

STEP 06 / OUTPUT PERIPHERALS

Add the two ribbon cables

Board OFF. Use whole straight-through FRC ribbons, with pin 1 aligned at each end. No individual LCD signal wires are needed.



Logical map only - header positions on your kit may differ.

Action	Exactly what to do
LCD ribbon	CND -> CNAD. Short JP16 and JP5. Leave graphic LCD connections unused.
LCD contrast	After final power-on, gently adjust POT3 if text is invisible. Use a screwdriver and no pressure; do not force or exceed the lab limit.
Buzzer ribbon	CNA -> CNA5. The buzzer is P0.11, through ribbon contact 8 and the existing onboard driver.
Keep unused	Motor and relay loads, LED ribbon CNA->CNA1, seven-segment ribbons, and graphic LCD. Do not try to use these together on CNA.
Polarity check	Ask staff to confirm the buzzer active level with their example. Default BUZZ_ACTIVE_HIGH=1; if active low, use 0, rebuild, and reflash.

LCD ribbon contact	GPIO	LCD function
1 / 2 / 3 / 4	P0.23 / P0.24 / P0.25 / P0.26	D4 / D5 / D6 / D7
5 / 6	P0.27 / P0.28	RS / EN

These mappings are derived from the source tables for a 1:1 ribbon. LCD power/contrast is onboard; add no separate LCD 5 V lead. R/W is not MCU-controlled; use the kit wiring as provided.

STEP 07 / JUMPERS AND CABLES

Set up the programming position

A jumper cap is SHORT when it joins the specified pins. OPEN means those pins are not joined. Ask staff if the label or cap orientation is unclear.

Item	FLASH position now	RUN position later
JP1	SHORT	SHORT: supplies sensor rail
JP2	OPEN	OPEN for this UART setup
JP3	SHORT: enter ISP	OPEN before run reset/power-on
JP5	SHORT	SHORT: LCD D7 path
JP6	SHORT	SHORT: normal 3.3 V board supply
JP16	SHORT	SHORT: select 16 x 2 LCD
SW21 (both DIP positions)	ON	OFF per supplied normal/UART0 run procedure

Leave all unrelated jumper groups in their existing lab-approved configuration. Do not remove the LPC1768 piggyback. **SW1 is RESET**; **SW21** is the programming handshake switch.

Connect the PC and power cables, still with power off

1. Plug the lab USB-to-serial adapter into the PC. Connect its serial end to the lab grey RS232 cross cable.

2. Connect the grey serial cable to **CN9: left female DB9** in the supplied board notes. Read the CN9 label to confirm.

3. Connect the separate lab-supplied power cable to **CN8: right male DB9**. Keep the power source switched off for now.

4. On Windows, open **Device Manager -> Ports (COM & LPT)**. Note the USB-serial port, for example COM4. Your number may differ.

5. Close serial terminals that could hold that COM port open. Keep W4 sensor VCC and W7 CN6 pin 2 disconnected.

These DB9 connectors are not interchangeable
CN9 uses RS232 levels via the onboard MAX3232. Do not plug a TTL UART device into CN9. Use only the lab power lead for CN8; a serial cable is not a power supply.

STEP 08 / KEIL ON THE PC

Save code, build, find the HEX

The board can remain OFF for this entire page. Best route: use a copy of the instructor-approved LPC1768 project that already builds on this kit.

1. Get the repository files on the PC. Copy the lab template project to a new folder such as **CollisionWarning**, then open its **.uvproj / .uvprojx** file in Keil.

2. Remove or exclude the template's old application file containing **int main(...)** from the build. Keep its startup file and **system_LPC17xx.c**. There must be only one application main function.

3. Copy this repository's **main.c** into that folder. In Keil's Project pane, right-click the source group -> **Add Existing Files to Group** -> choose main.c -> Add -> Close. It is fine to paste the complete listing into a new file and save it as main.c instead; do not include PDF line numbers.

4. Confirm target **NXP LPC1768** and board crystal **12.0 MHz**. Confirm the kit startup, CMSIS header **LPC17xx.h**, and system clock file are included. Leave **ENABLE_UART 0** for the first demo.

5. Open **Project -> Options for Target -> Output**; tick **Create HEX File**. For older lab profiles, the supplied settings list IROM1 start 0x0 / size 0x8000 and IRAM1 start 0x10000000 / size 0x8000. Use the instructor's approved profile; these are not the chip's full memory sizes.

6. Choose **Project -> Build Target** (commonly F7). Read Build Output. Continue only with **0 errors** and a created HEX file. Ask staff about warnings. An unused TTC warning with UART disabled does not prove hardware success or failure.

7. Find the generated **.hex** file in the configured output folder, often Objects. Record its full path and check its modification time. This is what you will select in Flash Magic, not main.c.

If you have no lab template
Project -> New µVision Project -> choose LPC1768. Use device support/startup selected by your Keil version and the kit's matching system file. In newer versions this may be through Manage Run-Time Environment. Ask staff to confirm this first build; the repository does not supply those vendor files.

The startup normally calls SystemInit() before main(). This program calls SystemCoreClockUpdate() in timer_init(). Do not add a second SystemInit() or rewrite PLL settings blindly. The 12 MHz crystal setting is not necessarily the CPU clock. Leave optional UART off: the source's 3 MHz core profile does not produce exact 9600 baud with the current divider.

STEP 09 / FIRST POWER-ON

Check power before sensor VCC

You are checking the power rails only. W4 (sensor VCC) and W7 (Node B to CN6 pin 2) must still be disconnected and isolated.

1. With power off, ask staff to inspect resistor placement, CN6 numbering, ribbons, programming jumpers, and the separate serial/power cables.

2. Ask staff to operate the multimeter on **DC volts**, with its black probe on GND. Never use resistance/continuity mode on a powered kit, and never place a meter in current mode across a supply.

3. Switch on the lab board supply. With JP6 short, verify the power rails using the table below. Avoid letting probe tips bridge adjacent pins.

4. If the readings are wrong, or anything heats/smells, switch off immediately. Do not connect sensor VCC or P2.1. Have staff investigate.

5. When supplies are correct, switch the board **OFF**. Keep the sensor VCC lead and P2.1 lead loose for the first flash. Proceed to page 11.

Measurement	Expected nominal result	Who checks
TP1 to TP3 (GND)	About +5 V	Lab staff / supervised measurement
TP2 to TP3 (GND)	About +3.3 V	Lab staff / supervised measurement
CN6 pin 25 to pin 26	About +5 V with JP1 short	Lab staff confirms pin orientation first
Breadboard +5 V rail to GND rail	About +5 V	Confirms W3/W1 rail routing

A staged first power-on is intentional
You have not yet connected the sensor supply or the MCU ECHO input. This prevents a rail mistake reaching the sensor/input before the basic power check. Do not skip directly from wiring to a full powered demo.

The reference describes removing JP6 before first assembling a piggyback board. That is a staff commissioning procedure for an unassembled kit. For this already assembled lab kit, keep JP6 short and do not remove the piggyback.

STEP 10 / FLASH MAGIC

Send the built program to the kit

Board OFF while selecting the HEX and changing cables/jumpers. Sensor VCC and P2.1 are still disconnected for this first programming stage.

Flash Magic field	Set / select
Device / interface	LPC1768 / None (ISP), as named in the supplied lab notes
COM port	Your Device Manager port, not an assumed COM1
Baud / oscillator	Lab-supported 115200 baud; crystal 12 MHz
Erase / verify	Erase blocks used by HEX before programming; verify after programming
HEX file	The newly built main.c application HEX from page 9
Advanced -> Hardware Config	DTR/RTS control RST/ISP; keep RTS asserted while COM is open
Hardware timing	T1 = 50 ms; T2 = 100 ms, from supplied notes

1. Confirm **JP2 open, JP3 short, JP6 short, SW21 both ON**. Keep JP1/JP5/JP16 short for the completed wiring.

2. After selecting the HEX with the kit off, power the kit on. Click **Start** in Flash Magic. Wait for erase, program, and verification to finish; read the success/error messages.

3. If unsuccessful, do not keep changing wires under power. Use the communication checks on page 15. The supplied lab sheet recommends High Speed Communications at 115200, while the ALS manual differs; use the lab-supported setting or try 9600/7200.

4. After successful verification, disconnect/close Flash Magic's serial session. Switch the board off. You will now change to RUN on page 12.

Do not erase it afterward
The sequence is **erase -> program -> verify -> run**. Erasing after a successful download removes the application. The source's phrase about clearing flash after each download must not be followed literally.

Flash Magic success means the program was transferred, not that sensor readings, timing, or physical wiring have been validated. Its ISP baud is separate from the optional application UART setting.

STEP 11 / VERIFY ECHO

Test the divider before P2.1

A lab instructor must check the ECHO high pulse. Keep W7 disconnected from CN6 pin 2 until the pulse check passes.

1. With power **OFF**, **open JP3** and set **SW21 both OFF** using the lab run procedure. Leave JP1/JP5/JP6/JP16 short and JP2 open.

2. Connect **W4: HC-SR04 VCC to the verified +5 V rail**. Check sensor GND, TRIG, ECHO-to-R1, and all three resistors. Keep the CN6 pin 2 end of W7 loose and isolated.

3. Place a flat target about **50 cm** in front of the sensor. Power on/reset. The flashed firmware now generates trigger pulses automatically even though its ECHO input is disconnected.

4. Ask staff to measure **Node B / row 20 relative to GND during an ECHO high**, using the instrument correctly. Expected peak is about **2.62 V** for a 5 V ECHO. Check it stays within the supplied 3.3 V limit for the intended input.

5. If no pulse is seen, first verify the program is running, P2.0 TRIG pulses, sensor power/ground, and target alignment. An idle multimeter reading of 0 V is **not** proof of correct division.

6. After staff confirm the pulse is safe, switch power **OFF**. Connect **W7 from D20 / Node B to CN6 pin 2 / P2.1**. Check it does not touch adjacent contacts.

7. Now power on/reset again. This is the first full sensor-to-MCU run. Continue to page 13.

SENSOR ERROR is expected during the disconnected-input test
The MCU cannot receive ECHO while W7 is loose. The LCD may show SENSOR ERROR / CHECK HC-SR04. Do not move W7 under power to try to clear it. Switch off, finish the validated connection, and restart.

If the buzzer is unexpectedly on during SENSOR ERROR, switch off and check its active polarity with staff. The program intends to silence it in that state; the board's actual active level must match BUZZ_ACTIVE_HIGH.

STEP 12 / FIRST OUTPUT

Read the LCD and hear the alarm

Use a flat book, board, or cardboard target facing the two transducers. Measure distance from the front of the sensor to the target.

1. Confirm RUN settings: **JP3 open, SW21 off**. All seven mandatory sensor wires are now connected. Power on; the firmware starts automatically. You do not need to press a PC Run button.

2. Keep a target at about **100 cm**. Wait for at least three valid readings and allow about one second for filtering/motion estimates to settle.

3. If the LCD is blank, adjust **POT3** gently within the lab's limit. If still blank, power off before inspecting CND->CNAD, JP16, and JP5.

4. Check that D roughly follows the target distance. Keep the target still: V should tend toward 0 and W toward 50.

5. Move to about **40 cm**, hold still, then to about **15 cm**. You should see WARNING, then CRITICAL with buzzer, assuming valid measurements and correct polarity.

Illustrative stationary-target display



Readout	Meaning
D	Filtered distance in centimeters; it may lag briefly after moving the target.
V	Estimated closing velocity in cm/s. Moving away is displayed as zero closing velocity.
W	Current warning boundary in cm, normally 50 at rest and up to 150 for approach.
State	SAFE / WARNING / CRITICAL. Only CRITICAL commands the buzzer on.
SENSOR ERROR	Three invalid readings cause a fault; three consecutive valid readings recover it. Startup also requires three valid readings.

STEP 13 / DEMONSTRATE

Run these tests in order

These are expected software behaviors to verify on the bench, not measured results. Keep hands and objects clear of connectors while powered.

Test	What you do	Expected behavior
1. Stationary safe	Hold a flat target at 100 cm for a few seconds.	D near 100; V settles near 0; W near 50; SAFE; buzzer off.
2. Warning	Move target to 40 cm and hold it.	After filtering settles: WARNING, W near 50; buzzer off.
3. Critical	Move target to 15 cm, keeping it in the beam.	CRITICAL; buzzer on after a valid close reading, once startup recovery has completed.
4. Clear critical	Move from 15 cm to 30 cm and hold.	Critical holds until filtered D exceeds 25 cm; then normally WARNING at 30 cm.
5. Clear warning	Move from 40 cm to 60 cm and hold.	At settled W=50, WARNING clears only above 55 cm; then SAFE.
6. Adaptive response	Approach slowly, then faster, from a valid distance such as 100 cm.	V becomes positive; W can rise, giving a larger warning boundary. A reading of 50 cm/s gives W=75 cm.
7. Missing echo	Power off. Disconnect W7 at CN6 pin 2 and isolate it. Power on.	SENSOR ERROR remains; startup cannot recover without valid samples. Buzzer is intended off.
8. Recovery	Power off, reconnect W7, power on with target at 100 cm.	Three consecutive valid readings permit normal classification.

For a running-fault demonstration, a powered W7 disconnection is not part of this beginner procedure. Out-of-range/no-reflection conditions can also produce invalid readings. Never rewire a live input just to test the counter.

Limits of what you can conclude

The valid range is configured as 2-400 cm, not a guarantee on all surfaces. The filter uses five readings; one or two invalid readings can leave old values visible. A missing reflection can mean no suitable target, not necessarily a broken sensor.

IF SOMETHING DOES NOT WORK

Find the failing stage first

Switch off before every wiring, cable, or jumper change. Keep a note of the exact error message; do not change several things at once.

Problem	Check in this order
No power / no LED	Lab power source -> CN8 power cable -> JP6 short -> supervised TP1/TP2/GND measurements.
Failed to open COM	Device Manager COM number -> driver/adapter recognized -> close serial terminal -> correct COM selected in Flash Magic.
Failed to autobaud	CN9 serial cable -> JP2 open / JP3 short / SW21 on -> lab high-speed setting -> 115200, then supported lower baud (9600/7200).
Still cannot communicate	Power off -> change USB port -> recheck new COM number. Lab replacement order: USB cable, RS232 cable, power supply, then kit.
LPC17xx.h missing	Install/select LPC1768 device/CMSIS support and use the lab template include paths. Do not remove the include to hide the error.
SystemInit or SystemCoreClock unresolved	Missing/mismatched kit startup or system_LPC17xx.c. Ask staff to restore the correct project files.
Duplicate main / duplicate symbols	Old demo main.c still included, or startup/system files included twice. Exclude duplicates from the build.
HEX flashed but nothing runs	Correct HEX timestamp -> JP3 open / SW21 off -> reset -> board supplies -> clock/startup match.
LCD blank / blocks only	POT3 contrast -> power off -> CND->CNAD -> JP16 and JP5 short -> complete firmware and clock/timing.
SENSOR ERROR persists	Flat target/alignment -> power off -> sensor labels/5 V/GND -> W5 TRIG -> divider and W7 at Node B -> staff checks pulses.
Buzzer silent in CRITICAL	State really CRITICAL -> power off -> CNA->CNA5 -> staff polarity test -> correct BUZZ_ACTIVE_HIGH -> rebuild and reflash.
Motor/relay acts / anything heats	Power off immediately; disconnect unused loads and ask staff to inspect CNA5 and the loaded program.

Optional UART: keep ENABLE_UART=0 until the core clock and divider are validated. The source's 3 MHz core profile makes the current integer divider produce about 11,719 baud, not the intended 9600. Serial logging is not required for the LCD/buzzer demo.

KEEP THIS PAGE BESIDE THE BOARD

Final connection and run checklist

Read the wire numbers together with page 6. This page is a recap; it does not replace the staged voltage checks on pages 10-12.

Check	Final connection / setting
[ ] Sensor supply	CN6 pin 25 -> +5 V rail -> sensor VCC; JP1 short.
[ ] Common ground	CN6 pin 26 -> GND rail -> sensor GND and R3 ground end.
[ ] Trigger	CN6 pin 1 / P2.0 -> sensor TRIG.
[ ] ECHO input	Sensor ECHO -> B10; R1 A10-A15; R2 B15-B20; R3 C20-GND.
[ ] ECHO output	D20 / Node B -> CN6 pin 2 / P2.1 only after safe-pulse check.
[ ] LCD	CND -> CNAD ribbon, pin 1 aligned; JP5 and JP16 short; POT3 adjusted.
[ ] Buzzer	CNA -> CNA5 ribbon, pin 1 aligned; loads disconnected; polarity checked.
[ ] PC / power	Serial -> CN9; lab power -> CN8. JP6 short.
[ ] RUN state	JP2 open; JP3 open; SW21 both off; JP1/JP5/JP6/JP16 short.
[ ] Program	Correct main.c built with LPC1768 startup/system; HEX programmed and verified.
[ ] Output	100 cm SAFE; 40 cm WARNING; 15 cm CRITICAL + buzzer; test with flat target.

Code and source trail

The next five pages contain the **complete current main.c**. Prefer the repository file to avoid PDF copying mistakes. For offline use, an exact main.c file is also attached inside this PDF; in a desktop PDF reader, look for the Attachments / paperclip panel and save it to your Keil project folder. Browser PDF viewers may not expose attachments.

Repository: github.com/ayatinkering/Ultrasonic-Collision-Warning-System
Guide based on supplied main.c, README.md, docs/design-notes.md, the ALS board pin-and-wiring reference, and LPC1768_HCSR04_Complete_Connection_Manual_REVISED.pdf (sections A-Q). Physical pin-1 orientation, adapter fit, supplies, and board performance remain for lab verification.

The supplied references cite the ALS user manual and MIT Manipal sheets. They disagree about some Flash Magic settings and contain pin-table uncertainties. This guide uses only the project's confirmed numbered signal assignments and flags the physical checks instead of inventing a board photograph. LCD/buzzer mappings assume a straight 1:1 ribbon. Do not copy unrelated peripheral-demo jumper settings into this build.

FIRMWARE APPENDIX / 1 OF 5

Complete main.c

Lines 1-50. Copy only code, not the grey line numbers. The embedded main.c attachment preserves exact bytes.

```
/* LPC1768 adaptive ultrasonic proximity warning
 * ALS-SDA-ARMCTXM3-01: TRIG P2.0, protected ECHO P2.1,
 * LCD D4..D7 P0.23..26, RS P0.27, EN P0.28, buzzer P0.11.
 * Build with Keil LPC1768 CMSIS device pack (LPC17xx.h + startup/system).
 * IMPORTANT: bench prototype, NOT an automotive safety device.
 */
#include "LPC17xx.h"
#include <stdint.h>
#include <stdio.h>

#define TRIG (1UL<<0)
#define ECHO (1UL<<1)
#define BUZZ (1UL<<11)
#define LCD_DATA (15UL<<23)
#define LCD_RS (1UL<<27)
#define LCD_EN (1UL<<28)
#define SAMPLE_US 70000UL
#define ECHO_TIMEOUT_US 30000UL
#define MIN_CM 2.0f
#define MAX_CM 400.0f
#define BASE_WARN_CM 50.0f
#define MAX_WARN_CM 150.0f
#define CRITICAL_CM 20.0f
#define HYST_CM 5.0f
#define VELOCITY_GAIN_S 0.5f
#define FILTER_ALPHA 0.35f
#define INVALID_LIMIT 3U
#define RECOVERY_LIMIT 3U
/* Set to 0 if your board's buzzer is active LOW after testing. */
#define BUZZ_ACTIVE_HIGH 1
/* UART output requires UART0 cable, terminal, and SW21 run position. */
#define ENABLE_UART 0

typedef enum { SAFE, WARNING, CRITICAL, SENSOR_ERROR } State;
static State state = SENSOR_ERROR;
static float history[5], prev_filtered, smooth_velocity;
static uint8_t history_count=0, history_pos=0, have_previous=0;
static uint8_t invalid_count=0, valid_count=0;
static uint32_t prev_sample_us=0;

/* Timer0 runs continuously at 1 MHz; unsigned subtraction handles wrap. */
static uint32_t micros(void) { return LPC_TIM0->TC; }
static void wait_us(uint32_t us) {
    uint32_t start=micros();
    while ((uint32_t)(micros()-start)<us) { }
}
static void timer_init(void) {
    SystemCoreClockUpdate();
    LPC_SC->PCONP |= (1UL<<1);           /* Power Timer0 */
    LPC_SC->PCLKSEL0 = (LPC_SC->PCLKSEL0 & ~(3UL<<2)) | (1UL<<2);
```

FIRMWARE APPENDIX / 2 OF 5

Complete main.c

Lines 51-100. Copy only code, not the grey line numbers. The embedded main.c attachment preserves exact bytes.

```
    LPC_TIM0->TCR=2;                    /* Reset timer */
    LPC_TIM0->PR=(SystemCoreClock/1000000UL)-1UL;
    LPC_TIM0->TCR=1;                    /* Start timer */
}
static void gpio_init(void) {
    LPC_PINCON->PINSEL4 &= ~((3UL<<0)|(3UL<<2)); /* P2.0/1 GPIO */
    LPC_GPIO2->FIODIR = (LPC_GPIO2->FIODIR | TRIG) & ~ECHO;
    LPC_GPIO2->FIOCLR=TRIG;
    LPC_PINCON->PINSEL0 &= ~(3UL<<22); /* P0.11 GPIO */
    LPC_GPIO0->FIODIR |= BUZZ;
    LPC_PINCON->PINSEL1 &= ~((3UL<<14)|(3UL<<16)|(3UL<<18)|
                             (3UL<<20)|(3UL<<22)|(3UL<<24));
    LPC_GPIO0->FIODIR |= LCD_DATA|LCD_RS|LCD_EN;
}
static void buzzer(uint8_t on) {
#if BUZZ_ACTIVE_HIGH
    if(on) LPC_GPIO0->FIOSET=BUZZ; else LPC_GPIO0->FIOCLR=BUZZ;
#else
    if(on) LPC_GPIO0->FIOCLR=BUZZ; else LPC_GPIO0->FIOSET=BUZZ;
#endif
}
/* HD44780 LCD: 4-bit bus, write-only, no R/W connection. */
static void lcd_nibble(uint8_t nibble) {
    LPC_GPIO0->FIOCLR=LCD_DATA;
    LPC_GPIO0->FIOSET=((uint32_t)(nibble&15U)<<23);
    LPC_GPIO0->FIOSET=LCD_EN;
    wait_us(2);
    LPC_GPIO0->FIOCLR=LCD_EN;
    wait_us(50);
}
static void lcd_write(uint8_t value, uint8_t data) {
    if(data) LPC_GPIO0->FIOSET=LCD_RS;
    else LPC_GPIO0->FIOCLR=LCD_RS;
    lcd_nibble(value>>4);
    lcd_nibble(value);
    if(value==1 || value==2) wait_us(2000);
}
static void lcd_cmd(uint8_t x) { lcd_write(x,0); }
static void lcd_text(const char *s) { while(*s) lcd_write((uint8_t)*s++,1); }
static void lcd_init(void) {
    wait_us(40000);
    LPC_GPIO0->FIOCLR=LCD_RS|LCD_EN;
    lcd_nibble(3); wait_us(5000);
    lcd_nibble(3); wait_us(200);
    lcd_nibble(3); lcd_nibble(2);
    lcd_cmd(0x28); /* 4-bit, 2-line */
    lcd_cmd(0x0C); /* Display on */
    lcd_cmd(0x06); /* Increment cursor */
    lcd_cmd(0x01); /* Clear */
}
```

FIRMWARE APPENDIX / 3 OF 5

Complete main.c

Lines 101-150. Copy only code, not the grey line numbers. The embedded main.c attachment preserves exact bytes.

```
/* Returns 1 for valid distance, 0 for missing/out-of-range echo. */
static uint8_t ultrasonic_read(float *cm) {
    uint32_t start, rise, width;
    /* Wait for prior echo to become low; avoid stale pulse. */
    start=micros();
    while(LPC_GPIO2->FIOPIN&ECHO)
        if((uint32_t)(micros()-start)>ECHO_TIMEOUT_US) return 0;
    LPC_GPIO2->FIOCLR=TRIG;
    wait_us(3);
    LPC_GPIO2->FIOSET=TRIG;
    wait_us(12);
    LPC_GPIO2->FIOCLR=TRIG;
    start=micros();
    while(!(LPC_GPIO2->FIOPIN&ECHO))
        if((uint32_t)(micros()-start)>ECHO_TIMEOUT_US) return 0;
    rise=micros();
    while(LPC_GPIO2->FIOPIN&ECHO)
        if((uint32_t)(micros()-rise)>ECHO_TIMEOUT_US) return 0;
    width=(uint32_t)(micros()-rise);
    *cm=(float)width/58.3f;
    return (*cm>=MIN_CM && *cm<=MAX_CM);
}
static float median_distance(float x) {
    float a[5], tmp;
    uint8_t i,j,n;
    history[history_pos]=x;
    history_pos=(history_pos+1U)%5U;
    if(history_count<5) history_count++;
    n=history_count;
    for(i=0;i<n;i++) a[i]=history[i];
    for(i=1;i<n;i++) {
        tmp=a[i]; j=i;
        while(j>0 && a[j-1]>tmp) { a[j]=a[j-1]; j--; }
        a[j]=tmp;
    }
    return a[n/2];
}
static float clampf(float x,float lo,float hi) {
    return x<lo?lo:(x>hi?hi:x);
}
static const char *state_name(State s) {
    switch(s) {
        case SAFE: return "SAFE";
        case WARNING: return "WARNING";
        case CRITICAL: return "CRITICAL";
        default: return "SENSOR ERROR";
    }
}
/* No single valid reading clears a latched sensor error. */
static void classify(float filtered,float raw,float threshold) {
```

FIRMWARE APPENDIX / 4 OF 5

Complete main.c

Lines 151-200. Copy only code, not the grey line numbers. The embedded main.c attachment preserves exact bytes.

```
151     float critical_observed=(raw<filtered)?raw:filtered;
152     if(state==SENSOR_ERROR && valid_count<RECOVERY_LIMIT) return;
153     if(critical_observed<=CRITICAL_CM) { state=CRITICAL; return; }
154     if(state==CRITICAL && filtered<=CRITICAL_CM+HYST_CM) return;
155     if(state==WARNING && filtered<=threshold+HYST_CM) return;
156     state=(filtered<=threshold)?WARNING:SAFE;
157 }
158 static void display(float d,float v,float warn) {
159     char line[17];
160     int dist=(int)(d+0.5f), vel=(int)(v+0.5f);
161     lcd_cmd(0x80);
162     if(state==SENSOR_ERROR) {
163         lcd_text("SENSOR ERROR    ");
164         lcd_cmd(0xC0); lcd_text("CHECK HC-SR04  ");
165         return;
166     }
167     snprintf(line,sizeof(line),"D:%3dcm V:%3d   ",dist,vel);
168     lcd_text(line);
169     lcd_cmd(0xC0);
170     snprintf(line,sizeof(line),"%-8s W:%3d  ",state_name(state),(int)(warn+0.5f));
171     lcd_text(line);
172 }
173 #if ENABLE_UART
174 static void uart_init(void) {
175     LPC_SC->PCONP|=(1UL<<3);
176     LPC_PINCON->PINSEL0=(LPC_PINCON->PINSEL0&~((3UL<<4)|(3UL<<6)))|
177                         (1UL<<4)|(1UL<<6);
178     LPC_SC->PCLKSEL0 &= ~(3UL<<6); /* UART0 PCLK = CCLK/4 */
179     LPC_UART0->LCR=0x83;
180     { uint32_t divisor=(SystemCoreClock/4UL)/(16UL*9600UL);
181       LPC_UART0->DLL=(uint8_t)divisor;
182       LPC_UART0->DLM=(uint8_t)(divisor>>8); }
183     LPC_UART0->FDR=0x10;
184     LPC_UART0->LCR=0x03;
185     LPC_UART0->FCR=0x07;
186 }
187 static void uart_puts(const char *s) {
188     while(*s) { while(!(LPC_UART0->LSR&(1UL<<5))) { }
189                 LPC_UART0->THR=(uint8_t)*s++; }
190 }
191 #endif
192 int main(void) {
193     uint32_t cycle,dt_us;
194     float raw=0, filtered=0, instantaneous=0, closing=0, warn=BASE_WARN_CM;
195     float ttc=0;
196 #if ENABLE_UART
197     char logline[120];
198 #endif
199     timer_init(); gpio_init(); buzzer(0); lcd_init();
200 #if ENABLE_UART
```

FIRMWARE APPENDIX / 5 OF 5

Complete main.c

Lines 201-245. Copy only code, not the grey line numbers. The embedded main.c attachment preserves exact bytes.

```
201     uart_init(); uart_puts("raw_cm,filtered_cm,closing_cm_s,threshold_cm,ttc_ms,state\r\n");
202 #endif
203     while(1) {
204         cycle=micros();
205         if(ultrasonic_read(&raw)) {
206             invalid_count=0;
207             if(valid_count<RECOVERY_LIMIT) valid_count++;
208             filtered=median_distance(raw);
209             if(have_previous) {
210                 dt_us=(uint32_t)(cycle-prev_sample_us);
211                 if(dt_us>0) {
212                     instantaneous=(prev_filtered-filtered)*1000000.0f/(float)dt_us;
213                     instantaneous=clampf(instantaneous,-300.0f,300.0f);
214                     smooth_velocity=FILTER_ALPHA*instantaneous+
215                                     (1.0f-FILTER_ALPHA)*smooth_velocity;
216                 }
217             } else { smooth_velocity=0; have_previous=1; }
218             prev_filtered=filtered;
219             prev_sample_us=cycle;
220             closing=clampf(smooth_velocity,0.0f,300.0f);
221             warn=clampf(BASE_WARN_CM+VELOCITY_GAIN_S*closing,
222                         BASE_WARN_CM,MAX_WARN_CM);
223             ttc=(closing>1.0f)?filtered/closing:0;
224             classify(filtered,raw,warn);
225         } else {
226             valid_count=0;
227             if(invalid_count<INVALID_LIMIT) invalid_count++;
228             if(invalid_count>=INVALID_LIMIT) {
229                 state=SENSOR_ERROR;
230                 history_count=0; history_pos=0;
231                 have_previous=0; smooth_velocity=0;
232             }
233         }
234         buzzer(state==CRITICAL);
235         display(filtered,closing,warn);
236 #if ENABLE_UART
237         snprintf(logline,sizeof(logline),"%d,%d,%d,%d,%d,%s\r\n",
238           (int)raw,(int)filtered,(int)closing,(int)warn,(int)(ttc*1000),
239           state_name(state));
240         uart_puts(logline); /* TTC field is milliseconds, not seconds */
241 #endif
242         /* Hold start-to-start period where possible; a timeout can overrun. */
243         while((uint32_t)(micros()-cycle)<SAMPLE_US) { }
244     }
245 }
```